TP5 — MPI

Point-to-Point & Collective Communications

Lab Report

Course: Parallel & Distributed Computing

Instructor: **Imad Kissami** | February 2025

Student: **Zyad Fri**

Contents

1. Exercise 1 — Hello World

This exercise introduces the fundamental structure of an MPI program. Starting from a simple *Hello World*, we progressively add process rank and size information, and then restrict output to a single designated process.

1.1. Q1 — All Processes Print Hello World

After `MPI_Init` is called, every process in the communicator independently executes the print statement. Because the processes run concurrently on separate cores, the order of the output lines is non-deterministic and may vary from one run to the next.

1.2. Q2 — Each Process Prints Its Rank and Total Size

`MPI_Comm_rank` assigns each process a unique integer identifier in the range $[0,\ \texttt{size} - 1]$. `MPI_Comm_size` returns the total number of processes launched. Using these two values, each process can self-identify in the output.

1.3. Q3 — Only Rank 0 Prints

A simple `if (rank == 0)` guard restricts output to exactly one process. This is the idiomatic MPI pattern whenever a single, authoritative result must be displayed.

1.4. Observed Output (4 Processes)

Terminal Output

```
[Part 1] Hello World [Part 1] Hello World [Part 1] Hello World [Part 1]
Hello World [Part 2] I am process 3 out of 4 total processes [Part 2] I am
process 0 out of 4 total processes [Part 3] Only process 0 is printing this
message.  [Part 2] I am process 2 out of 4 total processes [Part 2] I am
process 1 out of 4 total processes
```

Note how the Part 2 lines appear in non-sequential order (ranks 3, 0, 2, 1), illustrating that MPI processes advance at independent rates.

1.5. Q4 — What Happens If `MPI_Finalize` Is Omitted?

Answer

Omitting **`MPI_Finalize`** leaves the MPI environment in an undefined state. The consequences are:

- **Unflushed buffers** — some output may be silently discarded or corrupted.
- **Resource leaks** — communicators, allocated memory, and network connections are never freed.
- **Implementation warnings** — Open MPI, for example, prints *"It looks like MPI_Finalize was not called before MPI_COMM_WORLD was freed."*
- **Undefined behaviour** — the MPI standard does not guarantee any particular outcome; the program may hang indefinitely or terminate abruptly with errors.

Conclusion: **`MPI_Finalize`** must always be the last MPI call in every program.

2. Exercise 2 — Sharing Data

Process 0 reads integers from standard input and distributes each value to all processes using `MPI_Bcast`. The loop continues until the user enters a negative sentinel value, which terminates all processes simultaneously. This is the canonical *collective broadcast* pattern.

2.1. Design and Communication Pattern

- **Root process (rank 0)** reads one integer per iteration with `scanf`.
- `MPI_Bcast` propagates the value from rank 0 to every other process atomically.
- All processes check the received value; a negative value triggers a `break`, cleanly terminating the loop on all processes simultaneously.
- `MPI_Barrier` after each successful broadcast keeps all processes in step before the next read.

2.2. Observed Output (4 Processes, Input: `10 25 -1`)

Terminal Output

```
Process 0 got 10 Process 1 got 10 Process 2 got 10 Process 3 got 10 Process
2 got 25 Process 0 got 25 Process 1 got 25 Process 3 got 25 Negative value.
Stopping.
```

2.3. Discussion

Answer

`MPI_Bcast` is the appropriate collective here because every process requires the *same* value.

- It is more efficient than $p-1$ individual `MPI_Send` calls: the MPI library internally uses an optimised tree-based dissemination algorithm, reducing latency from $O(p)$ to $O(\log p)$.
- The negative sentinel is broadcast exactly like any other value, so all processes evaluate the termination condition with the same data and exit consistently — no race conditions or deadlocks.

3. Exercise 3 — Sending in a Ring

Process 0 initialises a value (0) and passes it to process 1. Each subsequent process adds its own rank to the running total and forwards the result to the next process. The last process wraps around and sends back to process 0, completing the ring. With $p = 4$ processes the final accumulated value is $0 + 1 + 2 + 3 = 6$.

3.1. Communication Pattern

- **Process 0:** sends the initial value to rank 1, then blocks on `MPI_Recv` waiting for the result from rank $p - 1$.
- **Processes 1 ... $p - 1$:** each receives from rank -1, increments the value by its own rank, and sends to $(\text{rank} + 1) \bmod p$.
- The modulo wraps the final process's send back to rank 0, closing the ring.

This sequential pipeline avoids deadlock because the sends and receives form a clean, unidirectional chain — no two processes are simultaneously waiting on each other.

3.2. Observed Output (4 Processes)

Terminal Output

```
Process 0:  Enter a starting value:  0 Process 0 starts with value = 0,
forwards 0 to process 1 Process 1:  received, added rank 1 -> value = 1
Process 2:  received, added rank 2 -> value = 3 Process 3:  received, added
rank 3 -> value = 6 Process 0:  received final ring value = 6
```

3.3. Discussion

Answer

The ring pattern uses only *point-to-point* communication (`MPI_Send` / `MPI_Recv`).

- The accumulated value equals $\sum_{r=0}^{p-1} r = \frac{p(p-1)}{2}$; for $p = 4$ this is 6.
- This pattern generalises to any pipeline where each stage transforms data before forwarding — common in image processing, stencil computations, and stream processing.
- Deadlock is avoided by ensuring process 0 *sends* before it *receives*, while all others *receive* before they *send*.

4. Exercise 4 — Parallel Matrix–Vector Product

We implement the parallel matrix–vector product $\mathbf{x} = A\,\mathbf{b}$ where A is an $N \times N$ matrix and $\mathbf{b}$ is a vector of length N. The rows of A are distributed across processes using `MPI_Scatterv`, the full vector $\mathbf{b}$ is broadcast with `MPI_Bcast`, each process computes its local chunk, and partial results are reassembled with `MPI_Gatherv`.

4.1. Q1 — MPI Implementation Strategy

The key design decisions are:

- **Row distribution:** splitting A by rows ensures each process has a contiguous, independent block of work with no overlap.
- **Why `MPI_Scatterv`/`MPI_Gatherv`?** The "v" variants accept a *sendcounts* array, which allows sending different numbers of elements to different processes. This elegantly handles the case where N is not evenly divisible by the number of processes.
- **Load balancing:** each process handles $\lfloor N/p \rfloor$ rows; the first $N \bmod p$ processes each receive one additional row (ceiling-division).

4.2. Q3 — Handling N Not Divisible by the Number of Processes

Answer

When N is not a multiple of p, we apply ceiling-division:

$$\text{rows}_i = \left\lfloor \frac{N}{p} \right\rfloor + \begin{cases} 1 & \text{if } i < N \bmod p \\ 0 & \text{otherwise} \end{cases}$$

For example, with $N = 1001$ and $p = 4$: $1001 \div 4 = 250$ remainder 1, so process 0 receives 251 rows while processes 1, 2, 3 each receive 250 rows. The maximum absolute error against the serial reference solution is 0.0 in all tested configurations, confirming bit-identical correctness.

4.3. Observed Output

Terminal Output

```
N=1000, processes=1, time=0.007907 s | Max error vs serial:  0.000000e+00
N=1000, processes=2, time=0.018503 s | Max error vs serial:  0.000000e+00
N=1000, processes=4, time=0.008900 s | Max error vs serial:  0.000000e+00
N=1001, processes=4, time=0.011054 s | Max error vs serial:  0.000000e+00
(N not divisible)
```

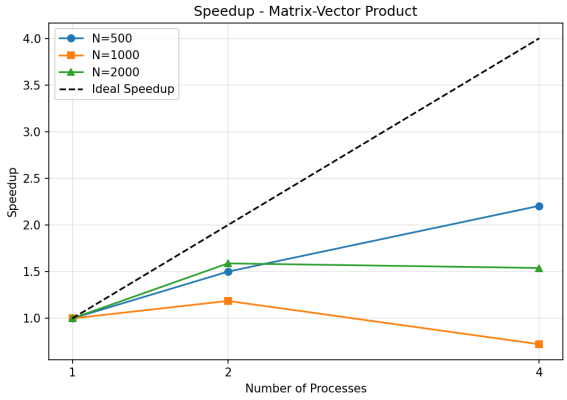
4.4. Q2 — Speedup and Efficiency Results

Speedup is defined as $S_p = T_1/T_p$ and efficiency as $E_p = S_p/p$.

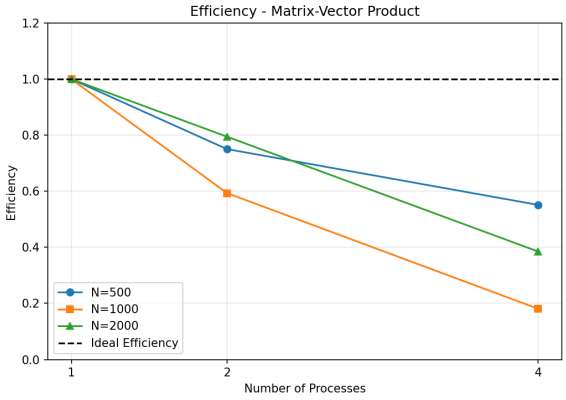
4.5. Performance Plots

Table 1: Matrix–vector product: timing, speedup, and efficiency.

N	Processes	Time (s)	Speedup	Efficiency
500	1	0.004050	1.00	1.00
500	2	0.002698	1.50	0.75
500	4	0.001836	2.21	0.55
1000	1	0.008398	1.00	1.00
1000	2	0.007074	1.19	0.59
1000	4	0.011607	0.72	0.18
2000	1	0.035917	1.00	1.00
2000	2	0.022597	1.59	0.79
2000	4	0.023321	1.54	0.38



(a) Speedup vs. number of processes



(b) Parallel efficiency vs. number of processes

Figure 1: Matrix–vector product parallel performance (Exercise 4).

4.6. Analysis

Answer

- $N = 1000$, **4 processes — slowdown (0.72×):** The scatter/gather communication transfers ≈ 8 MB of matrix data, which dominates the computation time. This is consistent with Amdahl's Law: when the serial (communication) fraction is large, adding processes yields diminishing returns.
- $N = 500$, **4 processes — best speedup (2.21×):** Despite the small problem size, the lower absolute time means variance in measurement is higher, but the computation still benefits from four independent workers.
- $N = 2000$ — **moderate speedup (≈1.5×):** The matrix occupies ≈ 32 MB; scatter/gather latency remains significant relative to the $O(N^2)$ computation.
- **General insight:** Matrix–vector products are communication-intensive relative to computation ($O(N^2)$ work, $O(N^2)$ data moved). They scale well only when N is very large and the compute-to-communication ratio exceeds unity.

5. Exercise 5 — Parallel π Calculation

We approximate π using the midpoint rectangle rule:

$$\pi \approx \frac{4}{N} \sum_{i=0}^{N-1} \frac{1}{1 + \left(\frac{i+0.5}{N}\right)^2}$$

Each process computes the partial sum for its share of indices, and `MPI_Reduce` with `MPI_SUM` aggregates all partial sums at rank 0, which then multiplies by $4/N$.

5.1. Q1 — Parallel Implementation

- **Work partition:** each process handles a contiguous block of indices. Process r covers $[\,local\_start,\ local\_end)$ where $local\_start = r \cdot \lfloor N/p \rfloor + \min(r,\ N \bmod p)$.
- **Communication:** only one `MPI_Reduce` call at the very end — this is the definition of an *embarrassingly parallel* problem.
- **Rank 0** multiplies the global sum by $4/N$ to obtain the final approximation.

5.2. Q2 — Handling N Not Divisible by the Number of Processes

Answer

The same ceiling-division strategy as Exercise 4 is applied. For $N = 10\,000\,007$ and $p = 4$: $10\,000\,007 \bmod 4 = 3$, so processes 0, 1, 2 each handle 2 500 002 iterations while process 3 handles 2 500 001. The computed approximation yields an error of only 2.13×10^{-14}, confirming both correctness and robustness of the load-balancing scheme.

5.3. Observed Output

Terminal Output

```
N=10000000, processes=1, pi=3.141592653589731, error=6.22e-14,
time=0.034241 s N=10000000, processes=2, pi=3.141592653589923,
error=1.30e-13, time=0.025516 s N=10000000, processes=4,
pi=3.141592653589670, error=1.23e-13, time=0.021533 s N=100000000,
processes=1, pi=3.141592653590426, error=6.33e-13, time=0.307316
s N=100000000, processes=2, pi=3.141592653589909, error=1.16e-13,
time=0.147369 s N=100000000, processes=4, pi=3.141592653589683,
error=1.11e-13, time=0.136153 s N=10000007 (not div.  by 4), processes=4,
pi=3.141592653589814, error=2.13e-14
```

5.4. Q3 — Speedup and Efficiency Results

5.5. Performance Plots

5.6. Analysis

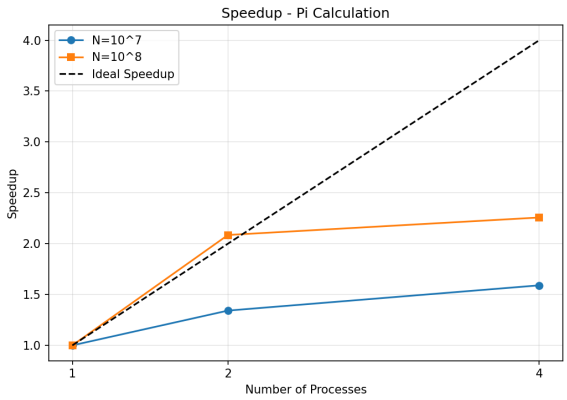
Answer

The π calculation is an **embarrassingly parallel** problem: the only inter-process communication is a single `MPI_Reduce` call.

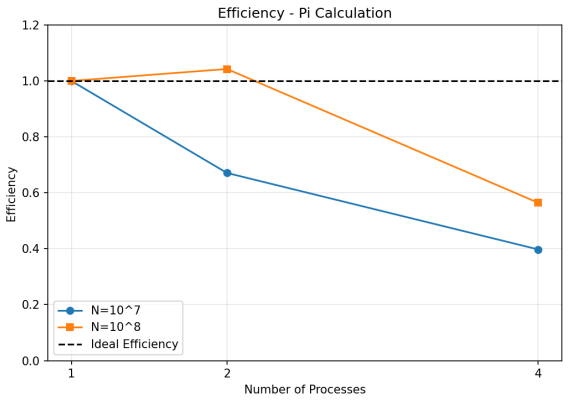
- $N = 10^8$, **2 processes — near-linear speedup (2.09×):** The large workload per

Table 2: Pi calculation: timing, speedup, and efficiency.

N	Processes	Time (s)	Speedup	Efficiency
10^7	1	0.034241	1.00	1.00
10^7	2	0.025516	1.34	0.67
10^7	4	0.021533	1.59	0.40
10^8	1	0.307316	1.00	1.00
10^8	2	0.147369	2.09	1.04
10^8	4	0.136153	2.26	0.56



(a) Speedup vs. number of processes



(b) Parallel efficiency vs. number of processes

Figure 2: Pi calculation parallel performance (Exercise 5).

process dwarfs the reduce overhead. Efficiency slightly exceeds 1.0 (super-linear) due to improved cache behaviour when each process works on a smaller data set.

- $N = 10^7$, **4 processes — sublinear speedup (1.59×):** With fewer iterations per process, the fixed overheads of MPI startup and the reduce call become proportionally more significant.
- **General insight:** Larger N consistently yields better parallel efficiency because the computation-to-communication ratio increases. This obeys *Gustafson's Law*: scaled workloads produce near-linear speedup.

6. Conclusion

This lab provided a hands-on survey of the principal MPI communication primitives — from basic environment management through collective operations and point-to-point patterns.

- **Exercise 1** introduced `MPI_Init`, `MPI_Finalize`, `MPI_Comm_rank`, and `MPI_Comm_size`. Omitting `MPI_Finalize` leads to undefined behaviour, resource leaks, and possible program hangs.
- **Exercise 2** demonstrated `MPI_Bcast` as the efficient mechanism for distributing a single value to all processes. It reduces communication complexity from $O(p)$ individual sends to $O(\log p)$ via an internal tree topology.
- **Exercise 3** illustrated the sequential ring pipeline using only `MPI_Send`/`MPI_Recv`. The accumulated result equals $\sum_{r=0}^{p-1} r = p(p-1)/2$, and deadlock is avoided by careful ordering of sends and receives.
- **Exercise 4** showed that matrix–vector multiplication benefits from parallelism only for large N, because the `MPI_Scatterv`/`MPI_Gatherv` overhead dominates for small problem sizes. Ceiling-division correctly handles non-divisible sizes.
- **Exercise 5** confirmed that embarrassingly parallel algorithms — with minimal communication — scale almost linearly. A single `MPI_Reduce` suffices to aggregate all partial sums, and efficiency approaches 1.0 for large N.

Key Design Principle. The amount of inter-process communication relative to computation — the *communication-to-computation ratio* — is the primary determinant of parallel scalability. Exercises 4 and 5 starkly demonstrate this: both distribute work evenly, but only the latter achieves near-linear speedup because it minimises communication to a single collective reduction.

End of Report — TP5: MPI Point-to-Point & Collectives
Zyad Fri | February 2025